DATABASE ASSESMENT BRIEF

Nicolò Rotella – GH1052515 – 09/04/2026

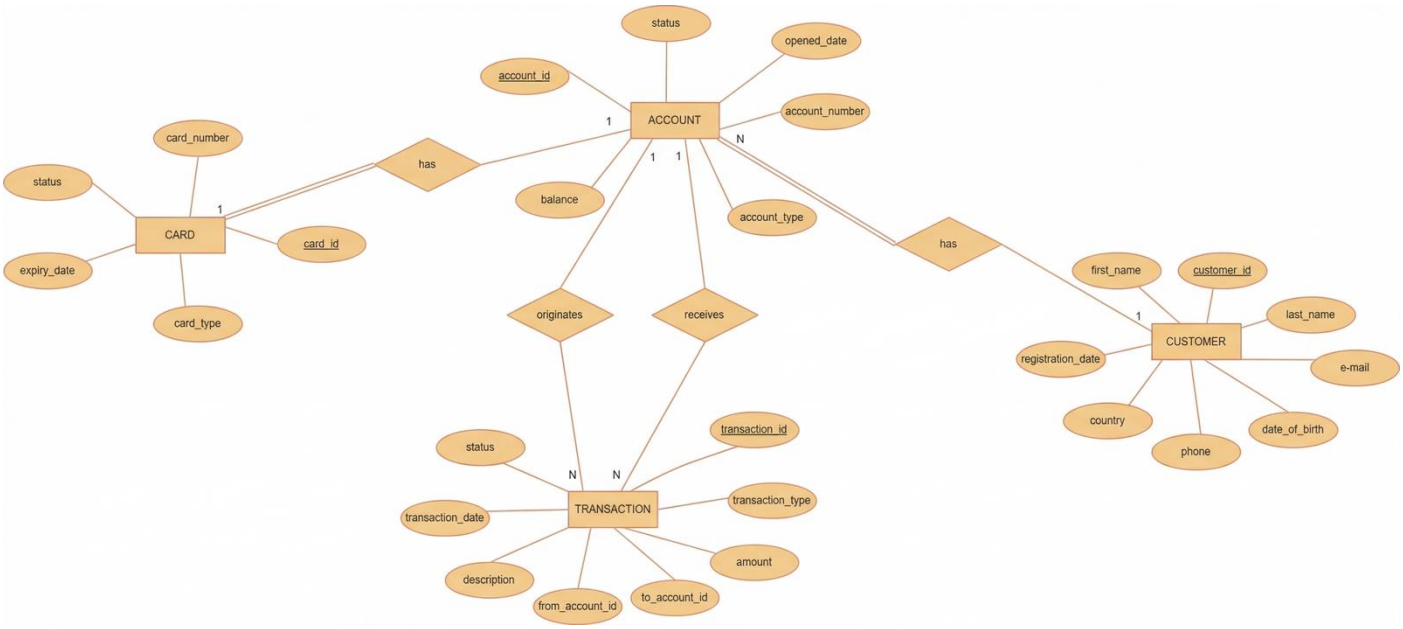
Video: https://youtu.be/-OFpcIlgIdU

GitHub repo: https://github.com/nicolo9840/ebank_database

Report

Nowadays the number of e-banks and their popularity have grown significantly. Moreover, an e-bank database must have complex relations and transfers, so due to these reasons I chose to develop an e-bank database. MySQL was chosen for its reliability and support for complex relational structures. I implemented the basic operations to manage clients, cards, accounts and transactions with standard queries and also stored procedures and transactions for more complex queries. Also, I added triggers to manage transactions and make sure the movements do not exceed the account limit (like a withdraw of 200€ from an account with balance 100€).

The ER diagram was designed with 4 tables and their fundamental attributes: Customer, Account, Card, Transaction.
I set a relation 1:N between Customer and Account, meaning that each customer can have many accounts. Also, I chose accounts to have at most 1 card, so with a relation 1:1 between Account table and Card table. Last I impose that an Account can do as many transactions as it wants, relation 1:N between Account and Transaction.
For the Transaction table I added the attributes from_account_id and to_account_id. In this way with a single table, it is possible to manage the 3 different operations that an e-bank account can perform: transfer from an account to another, withdraw and deposit. Indeed, a withdraw does not have an account of destination while a deposit does not have an originating account.
The ER diagram below illustrates the entities, attributes and relationships described above.



For the Customer table I impose registration_date to be NOT NULL and a DEFAULT of current date, to make sure it is filled automatically. Regarding phone and date_of_birth, they can be empty since sometimes they are not strictly necessary. Since email are used for important communication and to share contracts, Email data must be NOT NULL and UNIQUE.
For the Account table I chose the balance to be NOT NULL with a DEFAULT of 0.00 €, and a status NOT NULL with DEFAULT ‘Active’. Indeed, when a customer opens a new account it is standard to initialize it empty and active. Customer_id is the foreign key that links Account to Customer table.

For the Card table I set the status to be NOT NULL with a DEFAULT of 'Active'. card_number is set as UNIQUE and NOT NULL since it must uniquely identify each card, and expiry_date is NOT NULL as it is a mandatory attribute for any bank card. account_id is the foreign key to link the table with the Account table.
For the Transaction table I added a description attribute up to 250 characters (VARCHAR(250)), and a status attribute up to 20 characters that must be filled and with DEFAULT ‘Pending’. amount is NOT NULL since a transaction without an amount has no meaning, and transaction_date defaults to the current timestamp via NOW(). In this case there are 2 foreign keys both linked to the Account table which are from_account_id and to_account_id.

I created 2 triggers, one activates before every transaction insert stopping invalid movements, and the other after every transaction ensuring the balance is updated.

The first (check_balance_before_insert) performs a sequence of 2 checks. It starts looking if transaction_type is either a withdraw or a transfer; if so, it makes sure that from_account_id is not empty, otherwise it stops the transaction. The second check makes sure that the balance of the account is enough to perform the movement.

The second trigger (update_balance), that activates only if the transaction passes the first trigger, updates the balance following the type and amount of the transaction.

I designed 3 procedures:

- transfer_funds: it requires as inputs the account that originates the transfer, the account of destination and the amount. It opens a transaction and it checks the balance of origin. If it is not enough, the transaction is rolled back. Otherwise, it inserts the transaction and commits it.
- get_account_statement: it takes account_id, a starting date and an end date as inputs. The procedures returns all the transactions originated and received by that account in the specified period.
- freeze_account: only account_id as input. It checks the status of the account and if it is ‘Active’ it changes the status into ‘Frozen’, otherwise it returns an error.

The following screenshots demonstrate the proper functioning of some of the queries implemented in the database.

1. Transaction classification using a CASE expression based on transaction amount.

*<ebank> Script-4

```
SELECT transaction_id, amount,
CASE
    WHEN amount < 500 THEN 'Small'
    WHEN amount BETWEEN 500 AND 1000 THEN 'Medium'
    ELSE 'Large'
END AS transaction_size
FROM Transactions;
```

transactions 1

SELECT transaction_id, amount, CASE WHEN a | Enter a SQL expression to filter

	transaction_id	amount	transaction_size
1	1	3,000	Large
2	2	2,000	Large
3	3	335.5	Small
4	4	150	Small
5	5	200	Small
6	6	1,800	Large
7	7	850	Medium
8	8	900	Medium
9	9	500	Medium
10	10	500	Medium
11	11	500	Medium
12	12	500	Medium

2. Customer and account details retrieved using an INNER JOIN.

*<ebank> Script-4

```
SELECT c.customer_id, c.first_name, c.last_name, a.account_number, a.account_type, a.balance
FROM Customers c
INNER JOIN Accounts a ON c.customer_id = a.customer_id;
```

customers(+) 1 | Output

SELECT * FROM Accounts a | Enter a SQL expression to filter results (use Ctrl+Space)

	customer_id	first_name	last_name	account_number	account_type	balance
1	1	Paolo	Attico	IT84I020080950565756	Checking	2,650
2	1	Paolo	Attico	IT84I020000756567249	Savings	1,050
3	2	Sara	Maglia	IT86I500000758981644	Checking	0
4	2	Sara	Maglia	IT77I030000758981523	Savings	1,035.5
5	3	Ivan	Muschik	DE54I037780356987684	Checking	250

3. Account balances and statuses before and after calling the freeze_account and transfer_funds procedures.

*<ebank> Script-4

```
SELECT * from Accounts a ;
```

accounts 1

SELECT * from Accounts a | Enter a SQL expression to filter results (use Ctrl+Space)

	account_id	account_number	account_type	balance	status	opened_date	customer_id
1	1	IT84I020080950565756	Checking	2,150	Active	2025-03-14	1
2	2	IT84I020000756567249	Savings	1,550	Active	2025-03-10	1
3	3	IT86I500000758981644	Checking	0	Frozen	2024-10-22	2
4	4	IT77I030000758981523	Savings	1,035.5	Active	2025-06-29	2
5	5	DE54I037780356987684	Checking	250	Active	2026-01-20	3

*<ebank> Script-4

```
CALL freeze_account(5);
```

Statistics 1

Name	Value
Updated Rows	1
Execute time	0.019s
Start time	Thu Apr 09 14:08:55 CEST 2026
Finish time	Thu Apr 09 14:08:55 CEST 2026
Query	CALL freeze_account(5)

*<ebank> Script-4

```
CALL transfer_funds(2, 1, 500);
```

Statistics 1

Name	Value
Updated Rows	0
Execute time	0.066s
Start time	Thu Apr 09 14:07:53 CEST 2026
Finish time	Thu Apr 09 14:07:53 CEST 2026
Query	CALL transfer_funds(2, 1, 500)

*<ebank> Script-4

```
SELECT * from Accounts a ;
```

accounts 1

SELECT * from Accounts a | Enter a SQL expression to filter results (use Ctrl+Space)

	account_id	account_number	account_type	balance	status	opened_date	customer_id
1	1	IT84I020080950565756	Checking	2,650	Active	2025-03-14	1
2	2	IT84I020000756567249	Savings	1,050	Active	2025-03-10	1
3	3	IT86I500000758981644	Checking	0	Frozen	2024-10-22	2
4	4	IT77I030000758981523	Savings	1,035.5	Active	2025-06-29	2
5	5	DE54I037780356987684	Checking	250	Frozen	2026-01-20	3

Among the challenges faced building this database, one happened because I did not realize that when an account has just been opened it must have a balance of zero. If not, it can create problems with the correct functioning of the triggers.

I encountered another problem while creating the before insert trigger. In fact, it did not manage the case for withdraw and transfer with a NULL from_account_id, which is something unrealistic. For this reason, I implemented an explicit control for this scenario.

With this project for the first time I built a complete database from scratch and I found the process interesting and instructive. I had to think outside the box to solve some issues and I enjoyed the process of drawing the ER diagram first, and then dive into queries with increasing complexity. For sure there is room for improvement but after building it I feel much more comfortable with MySQL and database construction.